Strategies for Mapping OpenID Connect Identities to LinkedIn Public Profiles

The digital identity landscape has undergone a significant architectural shift as major professional platforms transition from proprietary authorization frameworks toward standardized protocols. LinkedIn's strategic move to adopt OpenID Connect (OIDC) represents a pivotal evolution in how third-party applications authenticate members and retrieve their professional data.[1] For systems architects and software engineers, the primary challenge in this new paradigm lies in the reconciliation of abstract, immutable identifiers—such as the OIDC subject claim—with the human-readable public profile URLs that define a member's professional presence.[2]

Historically, developers relied on a suite of proprietary OAuth 2.0 scopes, most notably r_basicprofile and r_liteprofile, to interface with the LinkedIn Profile API.[4] These legacy scopes permitted direct access to a member's vanityName, a unique string used to construct a profile link in the format www.linkedin.com/in/{vanityName}.[4] However, as of August 2023, LinkedIn deprecated these legacy scopes for new applications, mandating a transition to OIDC-based authentication.[6] This transition introduced a fundamental change in data delivery: where the Profile API once provided an array of descriptive profile fields by default, the OIDC userinfo endpoint prioritizes identity verification through a minimal set of standard claims.[1]

The primary task for modern developers is to identify the most efficient and sustainable mechanism for mapping an authenticated OIDC identity to a member's public profile link. This requirement is not merely a technical hurdle but a strategic necessity for applications that facilitate professional networking, recruiting, or identity verification. In the current API ecosystem, the most robust and officially sanctioned "best way" to achieve this mapping is through the integration of the "Verified on LinkedIn" product and its associated /identityMe endpoint.[3]

The Architecture of LinkedIn OpenID Connect

To understand the mapping process, one must first analyze the technical foundations of LinkedIn's OIDC implementation. OpenID Connect is an identity layer built on top of the OAuth 2.0 protocol, enabling applications to verify the identity of a member based on the authentication performed by an Authorization Server.[1] LinkedIn's implementation follows the standard OIDC discovery mechanism, providing configuration metadata via a well-known endpoint located at https://www.linkedin.com/oauth/.well-known/openid-configuration.[11]

The OIDC discovery document provides a roadmap for the authentication lifecycle, defining the endpoints necessary for authorization, token exchange, and user information retrieval. These endpoints are summarized in the following table:

Endpoint Type	URL	Functional Purpose
Issuer URL	https://www.linkedin.com/oauth	Identifies the provider and facilitates token validation.
Authorization Endpoint	https://www.linkedin.com/oauth/v2/authorization	Redirects the member to grant application access.
Token Endpoint	https://www.linkedin.com/oauth/v2/accessToken	Exchanges authorization codes for access and ID tokens.
UserInfo Endpoint	https://api.linkedin.com/v2/userinfo	Returns authenticated user claims based on scopes.
JWKS URI	https://www.linkedin.com/oauth/openid/jwks	Provides public keys for verifying JWT token authenticity.

11

The transition to OIDC simplifies the onboarding experience by eliminating the need for multiple API calls to establish basic member identity.[1] Upon a successful authentication flow, the application receives an ID Token—a JSON Web Token (JWT) containing security claims about the authentication event.[1] The most critical claim for mapping purposes is the sub (subject) claim, which serves as the unique, immutable identifier for the member within the context of that specific application.[1]

The Subject Claim as an Identity Anchor

The sub claim is the primary key for all future interactions with the member's data. It is important to note that this identifier is scoped to the developer application; using the same member's sub value with a different application will not yield the same results.[4] For internal database mapping, the sub value is frequently used to construct a Uniform Resource Name

(URN), typically formatted as urn:li:person:{sub_value}.[2]

While the sub claim provides a reliable anchor for identity, the standard OIDC profile scope does not natively include the member's public profile URL.[2] The data returned via the /v2/userinfo endpoint is limited to the following fields:

Field Name	**Description**	**Data Type**
sub	Subject Identifier (Unique User ID)	Text
name	Full name of the member	Text
given_name	Member's first name	Text
family_name	Member's last name	Text
picture	URL of the member's profile picture	Text
locale	Member's language and country locale	Text
email	Member's primary email address (if email scope is used)	Text
email_verified	Boolean indicating if the email has been verified	Boolean

Because the profileUrl is conspicuously absent from this standard claim set, developers must look toward specialized LinkedIn API products to bridge the gap between a logged-in session and a public-facing profile link.

The Definitive Solution: The IdentityMe Endpoint

The evolution of the LinkedIn developer platform in 2024 and 2025 has designated the /identityMe endpoint as the superior method for retrieving profile links.[3] This endpoint is part of the "Verified on LinkedIn" product suite and is specifically designed to provide trusted, verified profile information to partner platforms.[10]

The /identityMe API is a RESTful interface that retrieves comprehensive member profile details with the member's explicit consent.[3] Unlike the standard OIDC endpoint, which is optimized for high-speed authentication, /identityMe is optimized for profile display and verification use cases.[3]

Product Tiers and Data Accessibility

The "Verified on LinkedIn" product is structured into three tiers, each tailored to different developer needs and stages of application growth. Mapping a profile link is possible in all three tiers, though the breadth of additional data varies.

Tier	Availability	Primary Use Case	Key Mapping Features
Development	Immediate / Automated	Internal testing and prototyping.	Access to profileUrl for app admins only.
Lite	Requires Review (~1 week)	Production access for standard apps.	Access to profileUrl for all LinkedIn members.
Plus	Strategic Approval	Enterprise-scale verification and HR tech.	Access to profileUrl, workplace, and education.

To access this endpoint, the application must be provisioned with the "Verified on LinkedIn" product in the LinkedIn Developer Portal.[10] Once enabled, the application can request the r_profile_basicinfo scope during the OIDC authorization flow.[3]

Technical Execution of the IdentityMe Request

The mapping process begins with the construction of the 3-legged OAuth 2.0 authorization URL. It is recommended to request all required scopes simultaneously to minimize user friction and avoid multiple consent prompts.[16] A typical authorization request for mapping a profile link would include the following scopes: openid, profile, email, and r_profile_basicinfo.[14]

The redirection URL to LinkedIn follows this structure:

```HTTP
GET https://www.linkedin.com/oauth/v2/authorization?
  response_type=code&
  client_id={YOUR_CLIENT_ID}&
  redirect_uri={YOUR_REDIRECT_URI}&
  state={RANDOM_STRING}&
  scope=openid%20profile%20email%20r_profile_basicinfo
```

Upon successful authentication and consent, LinkedIn redirects the member to the specified URI with an authorization code. This code has an extremely short lifespan—approximately 20 seconds—and must be exchanged for an access token immediately.[20] The exchange is performed via a POST request to the token endpoint:

```Bash
curl -X POST 'https://www.linkedin.com/oauth/v2/accessToken' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=authorization_code' \
  -d 'code={AUTHORIZATION_CODE}' \
  -d 'redirect_uri={YOUR_REDIRECT_URI}' \
  -d 'client_id={YOUR_CLIENT_ID}' \
```

```
  -d 'client_secret={YOUR_CLIENT_SECRET}'
```

The resulting access token is then used to call the /identityMe endpoint. A critical requirement for this call is the inclusion of the LinkedIn-Version header, which ensures compatibility with the versioned API system.[13] As of late 2025, version 202510.03 is the standard for the Plus tier, while Development and Lite tiers should consult the latest release notes for the current version string.[3]

```bash
curl -X GET 'https://api.linkedin.com/rest/identityMe' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}' \
  -H 'LinkedIn-Version: 202510.03'
```

Analysis of the IdentityMe Response Payload

The response from the /identityMe endpoint provides the definitive mapping between the internal application ID and the member's public profile. The basicInfo object contains the profileUrl field, which is the "best way" to obtain the link requested by the user.[3]

A successful response follows this schema:

Field Path	Type	Description
id	String	Unique member identifier scoped to the application.
lastRefreshedAt	Timestamp	Indicates when the profile data was last pulled.
basicInfo.firstName	Object	Localized first name of the member.

basicInfo.lastName	Object	Localized last name of the member.
basicInfo.profileUrl	String	The public LinkedIn profile URL for the member.
basicInfo.primaryEmailAddress	String	The verified primary email address of the member.
basicInfo.profilePicture	Object	Metadata regarding the profile photo and its expiry.

3

The profileUrl returned by this API is typically a persistent redirect URL (e.g., https://www.linkedin.com/profile-thirdparty-redirect/...).[3] This mechanism is superior to constructing a URL from a vanityName because it remains valid even if the member changes their public handle, ensuring the mapping in the third-party application does not break over time.[22]

Legacy Approaches and Migration Context

While the /identityMe endpoint is the current standard, understanding the legacy methods is essential for maintaining older integrations or for developers who have access to the Marketing Developer Program. Prior to the OIDC transition, mapping was primarily achieved through the construction of vanity URLs based on the vanityName field.[4]

The Vanity Name Mapping Logic

In the legacy v2 API, a request to https://api.linkedin.com/v2/me would return the vanityName if the r_basicprofile scope was present.[4] The application would then perform a simple string concatenation to generate the profile link:

$$URL = \text{"www.linkedin.com/in/"} + \text{vanityName}$$

However, this approach is fraught with challenges in the modern ecosystem. Most new applications created after August 2023 are restricted to OIDC scopes and cannot obtain the

r_basicprofile permission through self-service.[5] Furthermore, the vanityName is a user-editable field. LinkedIn allows members to change their profile URL once every 30 days.[22] If an application caches the vanity-based URL, it risks directing users to a 404 error page if the member updates their profile handle.[22]

Comparative Scope Analysis

The transition from the v1 and v2 APIs to the OIDC framework necessitated a mapping of old permissions to new scopes. This mapping is summarized below:

Legacy Scope (v1/v2)	Modern OIDC Equivalent	Impact on Profile Mapping
r_basicprofile	r_profile_basicinfo (via IdentityMe)	vanityName replaced by profileUrl.
r_liteprofile	profile	Loss of access to profileUrl in base claims.
r_emailaddress	email	No change in functionality, updated retrieval method.
w_share	w_member_social	No impact on mapping.

6

Developers who encounter the "Not enough permissions to access: GET /me" or "Scope 'r_basicprofile' is not authorized" errors are typically attempting to use legacy logic with a modern OIDC application.[5] The remediation for these errors is to migrate the integration to use the openid profile scopes for authentication and the r_profile_basicinfo scope for profile link mapping via the /identityMe endpoint.[14]

Implementation Best Practices and Security

Successfully mapping a LinkedIn identity to a profile link requires more than just calling an endpoint; it requires a robust implementation strategy that addresses security, rate limiting,

and data persistence.

Enforcing Identity Uniqueness

A critical requirement for any application mapping LinkedIn identities is the validation of ID uniqueness. Developers must ensure that a single LinkedIn id (the value returned by /identityMe) is mapped to only one user account within their own system.[17] Failing to enforce this constraint creates a vulnerability where one LinkedIn account could potentially verify or take over multiple internal accounts.[17]

This uniqueness should be enforced at the database level using a UNIQUE constraint on the field storing the LinkedIn ID. When a member attempts to log in or link their account, the application should first check if the returned LinkedIn ID already exists in the system. If it does, and it is mapped to a different internal user, the application should trigger a security alert or display a clear error message to the user.[17]

Token Storage and Lifecycle Management

Access tokens granted by LinkedIn have a standard lifetime of 60 days, while refresh tokens typically last for one year.[16] To ensure a seamless user experience and maintain the profile mapping, applications must implement a secure token storage and refresh mechanism.

Tokens must never be stored in client-side storage like localStorage, sessionStorage, or unencrypted cookies, as this exposes them to Cross-Site Scripting (XSS) attacks.[14] Instead, tokens should be encrypted at rest in a secure backend database using algorithms such as AES-256-GCM.[14]

The refresh token allows the application to obtain a new access token without requiring the member to re-authenticate, provided the member has not revoked the application's permissions.[27] Implementing an automatic refresh cycle—perhaps triggered 10 days before the access token's expiry—ensures that the profile mapping data remains current and accessible for background processes or periodic syncs.

Managing Rate Limits and Caching

The /identityMe endpoint is subject to rate limits that vary based on the application's tier. For the Development and Lite tiers, the limits are generally structured as follows:

Limit Type	Threshold	Description
Application-level	5,000 calls / day	Maximum total calls across all users.

Member-level	500 calls / day	Maximum calls per individual member.

3

For enterprise partners in the Plus tier, these limits are customized based on the partnership agreement.[3] To optimize API usage and stay within these limits, applications should implement temporary caching of the profileUrl. Since profile data does not change frequently, caching the URL for 24 to 48 hours is often sufficient. Developers should always check the lastRefreshedAt timestamp returned by the API to determine the freshness of their cached data.[3]

Deep Insight: Identity vs. Professional Verification

The separation of the OIDC /userinfo endpoint and the /identityMe endpoint reveals a deeper strategic intent by LinkedIn. By decoupling authentication from professional data retrieval, LinkedIn is positioning itself not just as a social login provider, but as a specialized "Professional Identity as a Service" (IDaaS).[10]

Standard OIDC login is intended for low-friction entry, where the application only needs to know that the user U is the same user U from yesterday. However, the requirement for a public profile link signifies a transition from a simple user session to a professional interaction. This is why the profile link is gated behind a "Product" that requires terms of service acceptance and a review process for production access.[10]

This decoupling also serves as a privacy-preserving measure. By providing a redirect-based profileUrl through the "Verified on LinkedIn" program, LinkedIn can audit which third-party applications are actively sending traffic to member profiles. This prevents "silent scraping" where an application might harvest thousands of profile links through a background OIDC flow without ever presenting them to the user.

UX and Branding Considerations

Mapping a user to their LinkedIn profile often culminates in the display of that link or a "Verified on LinkedIn" badge within the third-party application's user interface. LinkedIn provides specific branding guidelines and assets to ensure a consistent experience across the web.[1]

Developers should use the official "Sign In with LinkedIn" button images, which are available in multiple formats and sizes.[1] When a member has been successfully mapped and verified via the /identityMe endpoint, the application is permitted to display a "Verified on LinkedIn" badge.[10] This badge serves as a trust signal, indicating to other users on the platform that the member's identity, workplace, or education has been confirmed by LinkedIn.[28]

Troubleshooting and Error Remediation

A successful implementation must account for the various error states that can occur during the OIDC flow or the subsequent API calls. The LinkedIn API utilizes standard HTTP status codes to communicate the nature of these errors.[3]

Status Code	Error Message	Common Root Cause	Remediation Path
400	Bad Request	Malformed URL or missing version header.	Verify LinkedIn-Version header and parameters.
401	Unauthorized	Expired or invalid access token.	Refresh the token or re-authenticate the member.
403	Forbidden	Missing scope r_profile_basicinfo.	Update authorization URL and re-request consent.
403	Forbidden	Product not enabled.	Add "Verified on LinkedIn" in the Developer Portal.
403	Forbidden	Non-admin user (Dev Tier).	Upgrade to Lite Tier for production usage.
426	Upgrade Required	Deprecated API version.	Update the LinkedIn-Version header value.

429	Too Many Requests	Rate limit exceeded.	Implement exponential backoff or caching.

3

One of the most common pitfalls is the "Invalid access token" error, which often stems from using a token generated for the OIDC flow to call a legacy v2 endpoint, or vice versa.[15] Developers should ensure that their token management logic clearly distinguishes between the requirements of the standard OIDC userinfo endpoint and the versioned REST APIs like identityMe.

Future Outlook: The Versioned API Ecosystem

The sunsetting of legacy Marketing versions (e.g., 202504) and the introduction of new, highly specific versions (e.g., 202510.03) indicates that LinkedIn is moving toward a more disciplined, time-bound API lifecycle.[13] For profile mapping, this means that the "best way" is no longer a static endpoint, but a versioned one.

Developers must stay appraised of the LinkedIn Release Notes to ensure their LinkedIn-Version headers remain current. The move toward versioning allows LinkedIn to iterate on the basicInfo object—for example, removing the headline field or adding primaryEmailAddress—without breaking older integrations that specify a previous version.[21]

Comprehensive Mapping Conclusion

The architectural evidence identifies the **"Verified on LinkedIn" /identityMe endpoint** as the definitive best practice for mapping an OIDC-authenticated user to their LinkedIn profile link in 2025. This method succeeds where others fail by providing a persistent, redirect-safe profileUrl that is integrated directly into the modern OIDC authorization flow via the r_profile_basicinfo scope.

To implement this effectively, developers must:

1. Provision the "Verified on LinkedIn" product in the developer portal to gain access to the necessary tiers (Lite or Plus for production).[10]
2. Incorporate the r_profile_basicinfo scope into the OIDC authorization request alongside standard openid and profile scopes.[14]
3. Utilize the LinkedIn-Version header in all requests to the versioned /identityMe endpoint to ensure compatibility and access to the latest field schema.[3]
4. Adopt a security-first approach to identity mapping by enforcing database-level uniqueness on the LinkedIn ID and securing OAuth tokens with at-rest encryption.[14]

This strategy not only satisfies the immediate requirement of linking a user to their profile but also aligns the application with LinkedIn's long-term vision of a verified, professional web where identity is both standard-compliant and trust-enriched.

Works cited

1. Sign In with LinkedIn using OpenID Connect - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/self-serve/sign-in-with-linkedin-v2
2. Sign In with Linkedin using OpenID connect - Stack Overflow, accessed on April 24, 2026, https://stackoverflow.com/questions/79448334/sign-in-with-linkedin-using-openid-connect
3. Profile Details API (/identityMe) - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/api-reference/identity-me
4. Profile API - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/shared/integrations/people/profile-api
5. Why Your LinkedIn API Returns 'Not Enough Permissions' Error - Kondo, accessed on April 24, 2026, https://www.trykondo.com/blog/fix-linkedin-permissions-error
6. Self-Serve v1 to v2 API Migration Frequently Asked Questions - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/self-serve/migration-faq
7. Login with LinkedIn | Supabase Docs, accessed on April 24, 2026, https://supabase.com/docs/guides/auth/social-login/auth-linkedin
8. Support LinkedIn OpenID Connect Scopes (profile, email, openid) - n8n Community, accessed on April 24, 2026, https://community.n8n.io/t/support-linkedin-openid-connect-scopes-profile-email-openid/112573
9. Understanding user profile and the userinfo endpoint - Auth0 Community, accessed on April 24, 2026, https://community.auth0.com/t/understanding-user-profile-and-the-userinfo-endpoint/7882
10. Verified on LinkedIn – Overview - LinkedIn | Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/overview
11. LinkedIn OAuth & OIDC endpoints - Logto, accessed on April 24, 2026, https://logto.io/oauth-providers-explorer/linkedin
12. Add LinkedIn Open ID Connect (OIDC) as a social connection - All providers | Clerk Docs, accessed on April 24, 2026, https://clerk.com/docs/guides/configure/auth-strategies/social-connections/linke

din-oidc
13. Verified on LinkedIn | Documentation | Postman API Network, accessed on April 24, 2026, https://www.postman.com/linkedin-developer-apis/linkedin-consumer-solutions/documentation/3uh4135/verified-on-linkedin
14. Upgrade to Lite Tier - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/guides/upgrade-to-lite-tier
15. Quickstart (Development Tier) - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/guides/quickstart
16. Plus Tier Quickstart - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/guides/plus-tier/quickstart
17. Implementation Guide - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/guides/implementation-guide
18. Authentication for Verified on LinkedIn APIs - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/api-reference/authentication
19. Authentication failed: Include valid openId scopes like profile, email - Stack Overflow, accessed on April 24, 2026, https://stackoverflow.com/questions/77362266/authentication-failed-include-valid-openid-scopes-like-profile-email
20. 3-Step Guide to Add LinkedIn OpenID Sign-In to Your App (2025 Edition) - DEV Community, accessed on April 24, 2026, https://dev.to/lovestaco/3-step-guide-to-add-linkedin-openid-sign-in-to-your-app-2025-edition-1mjh
21. Verified on LinkedIn – Release Notes - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/release-notes
22. What is my LinkedIn URL? - Typefully, accessed on April 24, 2026, https://typefully.com/blog/what-is-my-linkedin-url
23. Linkedin API v2: Receive vanityname on signup linkedin - Stack Overflow, accessed on April 24, 2026, https://stackoverflow.com/questions/57249920/linkedin-api-v2-receive-vanityname-on-signup-linkedin
24. Linkedin API: How to activate r_basicprofile as a permission? - Stack Overflow, accessed on April 24, 2026, https://stackoverflow.com/questions/57453831/linkedin-api-how-to-activate-r-basicprofile-as-a-permission
25. How to format your LinkedIn URL - FidForward, accessed on April 24, 2026,

https://fidforward.com/blog/linkedin_url_format/
26. How to get permission to use LinkedIn endpoint https://api.linkedin.com/v2/me : r/linkedin - Reddit, accessed on April 24, 2026, https://www.reddit.com/r/linkedin/comments/1ai18bb/how_to_get_permission_to_use_linkedin_endpoint/
27. Frequently Asked Questions | LinkedIn Developer Network, accessed on April 24, 2026, https://developer.linkedin.com/support/faq
28. LinkedIn launches Verified on LinkedIn to improve online trust and authenticity, accessed on April 24, 2026, https://news.linkedin.com/2025/Verified_on_LinkedIn
29. Verification Report API (/verificationReport) - LinkedIn - Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/consumer/integrations/verified-on-linkedin/api-reference/verification-report
30. The pros and cons of the LinkedIn verification | Ready for Social - ReadyForSocial, accessed on April 24, 2026, https://readyforsocial.com/2025/04/09/the-pros-and-cons-of-the-linkedin-verification-2/
31. People Typeahead API - LinkedIn | Microsoft Learn, accessed on April 24, 2026, https://learn.microsoft.com/en-us/linkedin/marketing/community-management/shares/posts-atmention-search-api?view=li-lms-2026-04